ZeroDEX
SMART CONTRACT AUDIT

ZOKYO.

April 22th, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges.

TECHNICAL SUMMARY

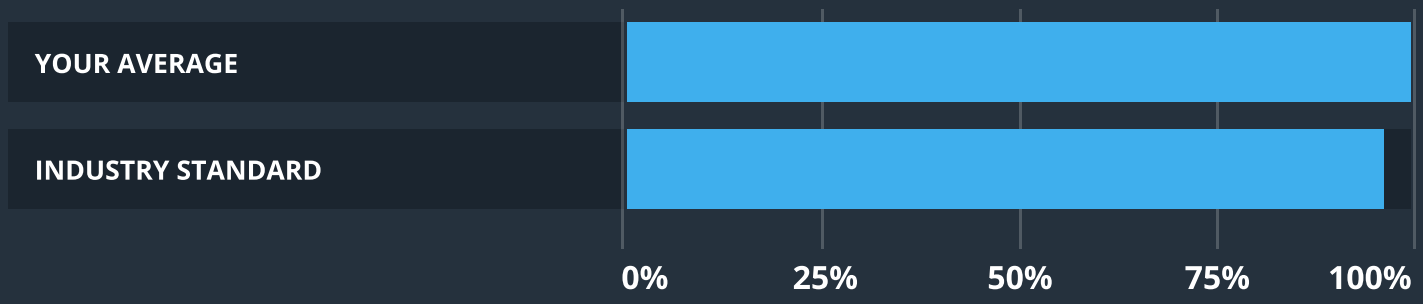
This document outlines the overall security of the ZeroDex smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the ZeroDex smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



The testable code is 100%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the ZeroDex team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Executive Summary 4
Structure and Organization of Document 5
Complete Analysis 6
Code Coverage and Test Results for all files 13

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the ZeroDex repository.

Repository - https://github.com/ZeroSwapLabs/zerodex-smart-contracts

Commit id - 5e882b4eaeca8c402bec1b6a6bc06d8c70102072

Last commit reviewed - b4d063a0c889525ffbf06e5b9bdf22ae36d26936

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of ZeroDex smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

EXECUTIVE SUMMARY

There were no critical issues found during the audit. All the mentioned findings may have an effect only in case of specific conditions performed by the contract owner.

The findings during the audit have a slight impact on contract performance or security, and can affect the further development and can influence some.

Nevertheless, all findings were successfully fixed by the ZeroDEX team.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the contract in such a way that funds may be lost, allocated incorrectly, or otherwise result in a significant loss.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

HIGH | RESOLVED

Absent framework.
No framework (Truffle or Brownie) and additional migrations/scripts - it affects further development and deployment processes.

Recommendation:
Provide the initial framework for the contracts set

HIGH | RESOLVED

No tests.
Absent test coverage. Formal logic is not checked, so currently it cannot guarantee the correctness of contracts functionality.

Recommendation:
Provide the initial test coverage for the formal logic of the contracts

MEDIUM | RESOLVED

transfer() instead of call()

EthDexManager.sol, line 210
transfer() for Eth is considered as unsafe (because of Istanbul Ethereum update) and should be replaced with call(). Especially because of missing restrictions to non-contract accounts.

Recommendation:
Change transfer() to call().

MEDIUM | RESOLVED

Use safeTransfer() and safeTransferFrom().

BaseDexManager.sol - line 96
BaseDexManager.sol - line 150
EthDexManager.sol - line 143
BaseDexManager.sol - line 130
EthDexManager.sol - line 120

Recommendation:
Use the safeTransfer and safeTransferFrom from SafeERC20 library.

LOW | RESOLVED

Extra code.

RuleEngine.sol, Imports Misc library with today() function which is repeated in the code (RuleEngine.sol, line 65)

Recommendation:
Remove extra code.

LOW | RESOLVED

No zero address check

RuleEngine.sol, line 37 - no check for zero addresses. It is mentioned as an issue, because there is no visible deployment script, so there is a place for mistakes. It is also important, since there is no interface for StakingContract changing.

Recommendation:
Use for example Misc.zeroOrContract().

LOW | RESOLVED

Incorrect math.

RuleEngine.sol, line 95. The function availableTrades() returns uint. Also, solc 0.8 has included safe math, so, return operation with unappropriate result of the subtraction will be reverted. Though, per the documentation for the function, it should return the amount of trades available. But the contract contains the functionality for manipulation with the stakes per trade calculation, so we can easily get the underflow (which will cause the revert).

Recommendation:
So the recommendation is to fix the statement and add “return 0” in case if “tradesBasedOnStake” will become less than number of trades per day.

INFORMATIONAL | UNRESOLVED

Extra contract.

DexManager has no functional role. It just has the same functionality as the EthDexManager, but adds more weight to the contract, more gas fees for initializing and can be completely replaced with the EthDexManager.

Recommendation:
Extra contract which can be reduced.

INFORMATIONAL | RESOLVED PER CONVERSATION

Max allowance instead of required.

BaseDexManager.sol, line 135
EthDexManager.sol, line 125
The contract provides the approval for the max amount instead of the required one. It is not safe from the funds flow point of view and creates a possibility to exploit the reentrancy in not safe calls from the contract. The issue is marked as low, because it does not affect the user, but only the manager contract and is covered in existing methods by appropriate checks.

Recommendation:
Provide approve for the amount needed instead of max.

	BaseDexManager	EthDexManager	RulesEngine
Re-entrancy	Pass	Pass	Pass
Access Management Hierarchy	Pass	Pass	Pass
Arithmetic Over/Under Flows	Pass	Pass	Pass
Unexpected Ether	Pass	Pass	Pass
Delegatecall	Pass	Pass	Pass
Default Public Visibility	Pass	Pass	Pass
Hidden Malicious Code	Pass	Pass	Pass
Entropy Illusion (Lack of Randomness)	Pass	Pass	Pass
External Contract Referencing	Pass	Pass	Pass
Short Address/ Parameter Attack	Pass	Pass	Pass
Unchecked CALL Return Values	Pass	Pass	Pass
Race Conditions / Front Running	Pass	Pass	Pass
General Denial Of Service (DOS)	Pass	Pass	Pass
Uninitialized Storage Pointers	Pass	Pass	Pass
Floating Points and Precision	Pass	Pass	Pass
Tx.Origin Authentication	Pass	Pass	Pass
Signatures Replay	Pass	Pass	Pass
Pool Asset Security (backdoors in the underlying ERC-20)	Pass	Pass	Pass

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by ZeroDEX team

As part of our work assisting ZeroDEX in verifying the correctness of their contract code, our team was responsible for checking the test coverage and correctness of existing tests for ZeroDEX contracts.

Based on the tests reviewed, Zokyo team conludes that ZeroDEX contracts have a sufficient test coverage for high level of security.

```
  Contract: DexManager: executeSwapBuyEthForRecipient with buyAmount
Warning: All subsequent Upgrades warnings will be silenced.

    Make sure you have manually checked all uses of unsafe flags.

    √ should revert if recipient doesn't send enough tokens for guaranteePrice * sellAmount (144199 gas)
    √ should execute if recipient has at least enough tokens for guaranteePrice * sellAmount (186842 gas)
    √ should return any extra sellToken amount that was not sent to the recipient (192398 gas)

  Contract: DexManager: executeSwapForRecipient with buyAmount
    √ should revert if recipient doesn't send enough tokens for guaranteePrice * sellAmount (174755 gas)
    √ should not revert if recipient has at least enough tokens for guaranteePrice * sellAmount (204120 gas)
    √ should return any extra sellToken amount that was not sent to the recipient (209476 gas)

  Contract: DexManager: executeSwapSellEthForRecipient with buyAmount
    √ should revert if recipient sends exactly sellAmount ETH when slippage is high (112121 gas)
    √ should execute if recipient has exactly enough ETH for guaranteePrice * sellAmount (159121 gas)
    √ should not allow sending anything more than maxSellAmount ETH (112121 gas)
    √ should not allow sending anything less than maxSellAmount ETH (112121 gas)

  Contract: DexManager: deposit
    √ should update the deposits array (64132 gas)

  Contract: DexManager: executeSwapBuyEthForRecipient
    √ should revert if called from a non-manager account (30414 gas)
    √ should revert if protocolFee does not match msg.value (60034 gas)
    √ should execute the trade (194897 gas)
    √ should emit a SwapExecuted event (182606 gas)
    √ should revert if the recipient has not staked (48007 gas)
    √ should revert if the recipient does not have enough tokens for the trade (139252 gas)
    √ should revert if the recipient has not approved enough tokens for the trade (97985 gas)

  Contract: DexManager: executeSwapForRecipient
    √ should revert if called from a non-manager account (30743 gas)
    √ should revert if protocolFee does not match msg.value (60363 gas)
    √ the user should not be charged any fees for the trade (189072 gas)
    √ should execute the trade (184872 gas)
    √ should emit a SwapExecuted event (184872 gas)
    √ should reduce the allowance for the dex manager contract (184872 gas)
    √ should reduce the available trades on the RuleEngine (189480 gas)
    √ should revert if the recipient has not staked (48336 gas)
    √ should revert if the recipient does not have enough tokens for the trade (96139 gas)
    √ should revert if the recipient has not approved enough tokens for the trade (98048 gas)
```

```
Contract: DexManager: executeSwapSellEthForRecipient
  √ should revert if called from a non-manager account (29890 gas)
  √ should revert if protocolFee does not match msg.value (59510 gas)
  √ the user should not be charged any fees for the trade (159565 gas)
  √ should update the user's available deposit (155365 gas)
  √ should execute the trade (155365 gas)
  √ should emit a SwapExecuted event (155365 gas)
  √ should update the deposits array (155365 gas)
  √ should revert if the user has not deposited enough funds for the trade (90392 gas)

Contract: DexManager: init
  √ should set the correct ruleEngine address (6866043 gas)

Contract: DexManager: withdraw
  √ should update the deposits array (102813 gas)
  √ should send the ETH to the user (102813 gas)

Contract: RuleEngine: assertTrade
  √ should only be able to be called by the DexManager (132066 gas)
  √ should revert if called by any address that's not the DexManager (25498 gas)
  √ should revert if called by any other contract (142355 gas)
  √ should revert if the stake threshold has not been reached (135188 gas)
  √ should revert if the tradeCap has been exceeded (208832 gas)
  √ should update tradesPerDay for the user (147054 gas)

Contract: RuleEngine: availableTrades
  √ should return 0 if the stake threshold has not been reached (96101 gas)
  √ should return the correct number of available trades when no trades have been made (166465 gas)
  √ should return the correct number of available trades after a trade happens (217406 gas)
  √ should reset the available trades the next day (217406 gas)

Contract: RuleEngine: stakePerTrade
  √ should update the stakePerTrade (30336 gas)

Contract: RuleEngine: DexManagerContract
  √ should set the DexManager (64150 gas)
  √ should revert if calling setDexManager from a non-admin address (59882 gas)
  √ should revert if trying to set a non-contract DexManagerContract (62628 gas)
  √ should revert if calling setDexManager with a 0 address (62376 gas)

Contract: RuleEngine: init
  √ should deploy a new ruleEngine and set the staker contract address (4291268 gas)
  √ should set the deployer address as admin (3760593 gas)

Contract: RuleEngine: tradeCap
  √ should update the tradeCap (30307 gas)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
RuleEngine\	100	100	100	100	
RuleEngine.sol	100	100	100	100	
access\	100	100	100	100	
AccessControl.sol	100	100	100	100	
dex\	100	100	100	100	
BaseDexManager.sol	100	100	100	100	
DexManager.sol	100	100	100	100	
Erc20DexManager.sol	100	100	100	100	
EthDexManager.sol	100	100	100	100	
All files	100	100	100	100	

We are grateful to have been given the opportunity to work with the ZeroDex team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the ZeroDex team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.